

Neutral accents / per-event velocity — scope

Goal. A Tidal-native, output-neutral way to specify accented drum patterns from the live-coding/generative tool. The *same* accent spec should render on MIDI→Ableton, OSC→SuperDirt, and CV/gate→modular. Two mandates:

1. **Strict superset of Tidal.** Don't duplicate or change how Tidal works; a Tidal user's muscle memory should apply verbatim.
2. **Don't make our own eDSL ugly** in the name of abstracting all three outputs.

Headline: the neutral quantity is *per-step strength* (0..1) — already the genre **StepProfile**, already Balistes' accent bit, and exactly Tidal's **gain**. Each backend renders it natively (MIDI velocity, Dirt amp, modular accent). We are not inventing a control; we are making one existing control flow.

What already works (the de-risk)

- **The dispatcher already applies per-event velocity to drum kits.** `Tidal/Dispatcher.purs` — `MidiNote` (line ~320) and `MidiDrumKit` (line ~359) both do `case Map.lookup "vel" params of ... param7bit ...`, overriding the binding/kit default velocity per event. `consumesVel` (line ~621) returns true for both. So `# vel` → MIDI note-on velocity is **done** on the MIDI leg, including drums.
- **The live-coding cell path already samples #-params.** `tidal_voice.erl` `install_from_spec(Name, PatStr, ParamSpecs)` installs a main pattern plus `# key pat` param patterns, samples each per step, and passes a `params` map to `dispatch_event`. Param specs are parsed as **text** over the WS verb, so they bypass PureScript's type checker.
- **Balistes is the precedent.** `balistes_voice.erl` renders the Grids per-step `{Inst, Accent}` to `vel(90)/vel_accnt(127)` MIDI velocity out to the FH-2 — i.e. strength→velocity→modular already ships. We mirror it; we do not route transcribed patterns through it (Balistes plays Grids' own 5x5 map, not arbitrary patterns).

So the rendering leg (per-event velocity to MIDI/FH-2) is **proven and present**.

What's missing (the actual work)

1. **Author surface — # won't compose with a drum pattern.** `drum "cga"` yields a `DrumPart` whose body is `Pattern DrumHitRef (Calypso/Prelude.purs ~line 421)`, not a `ControlPattern (Pattern ValueMap)`. So `drum "cga" # gain "..."` does not typecheck — you cannot *write* an accented drum cell in PureScript, even though the dispatcher would honour it. **This is the core change** (and the superset prize).
2. **Compiled-session parts don't carry per-event params.** Our generator emits a compiled `Calypso.Generated.Session` played via `play-piece`. Compiled parts are `Pattern note / Pattern DrumHitRef` with no `ValueMap`, so the walker has no params to sample → dispatch falls back to defaults. (The live `install_from_spec` path *does* carry params; the compiled path does not — yet.) **Open question to confirm first:** does the `SessionWalker` dispatch `PrimActions` directly,

or re-serialize parts into `install_from_spec` specs? That decides whether (1) alone suffices or whether the walker also needs to sample a ValueMap body.

3. **Neutral control name.** The dispatcher recognises `vel` (absolute 0..127). For neutrality + superset we want `gain` (0..1, Tidal's universal loudness): MIDI `velocity = round(gain*127)` (a curve later if needed), Dirt `amp`, modular accent. Keep `vel` as the absolute power-user override. One small helper in the param lookup + add `gain` to `consumesVel`.
4. **Modular accent leg.** `GateDrumKit` (Dispatcher ~line 371) fires a fixed gate; it does not yet read `gain/vel`. Modular accent = an accent gate or a velocity CV via es9-daemon. Needs es9-daemon support → **defer to Phase 2**.
5. **SuperDirt leg.** No Dirt PrimAction is wired (57120 collides with es9-daemon; rig is MIDI-first). `gain` → Dirt `amp` when/if Dirt is wired → **defer to Phase 3**.
6. **The lowering.** `calypso .../Tarot/Lower.purs` must emit each drum voice's `StepProfile` strength as a `# gain "..."` param alongside the tokens.

Surface design (the heart) — drums as ControlPatterns

Make `drum "cga ~ cga"` yield a **ControlPattern** (sets a sound/hit-token control, e.g. `s`), exactly like Tidal's `s "..."`. Then:

```
on vDrums kit (drum "cga ~ cga cga" # gain "1 ~ 0.6 0.9")  -- composes
natively
on vDrums kit (every 4 (# gain "1.1 0.5") (drum "cga*8"))  --
every/jux/off all apply
```

- `drum` stays as sugar (preserves the GM token vocabulary + kit binding), but underneath it is a vanilla ControlPattern → **every Tidal combinator works** and `#` composes. This *removes* a bespoke type rather than adding one.
- The `kit` stops being a pattern type and becomes a **binding** that reads the per-event map (token + `gain/vel`) and renders to the device. Dispatch already does the velocity half.
- **Backward compatible:** `on vDrums qd1 (drum "bd ~ sn")` (no `#`) is just a ControlPattern with no gain → unchanged behaviour. `every 8 rev (drum "...")` in `Sessions/Fugue/Full` keeps working (combinators apply to ControlPatterns).

Decision to confirm: keep the name `drum` (kit-token vocabulary, discoverable) vs reuse Tidal's `s`. Recommend **keep `drum`** as a thin alias for the superset feel without losing the kit vocabulary.

Neutral mapping (one knob → three renderings)

<code>gain</code> (0..1)	MIDI (Ableton + FH-2)	SuperDirt	Modular (gate/CV)
0	(rest / vel 0)	amp 0	no gate
0.5	velocity ≈ 64	amp 0.5	accent off
1.0	velocity 127	amp 1.0 (unity)	accent on / vel-CV high

`vel "0..127"` remains available as an absolute override (what the dispatcher recognises today).

Phasing

- **Phase 0 — verify rendering (no code)**. Over the raw WS, bind a drum kit and send `cga ~ cga # vel "120 0 70"`; confirm velocity reaches Ableton (drum-rack dynamics / MIDI monitor). Proves the dispatch leg end-to-end before touching types.
- **Phase 1 — surface + MIDI + lowering (the 80%)**. (a) `drum` → `ControlPattern`; adjust the `On/DrumKit` instance so `on vDrums kit (controlpat)` typechecks; keep back-compat. (b) Confirm!x the compiled-path param flow (open question #2); make compiled Parts carry a `ValueMap` body and the walker sample it (if needed). (c) Recognise `gain` (0..1) → velocity in the dispatcher; add to `consumesVel`. (d) Lowering emits `StepProfile` strength as `# gain "..."`. → Accented drums to Ableton + FH-2 on every genre. Testable by ear.
- **Phase 2 — modular accent**. `GateDrumKit` honours `gain` → accent gate / vel CV via `es9-daemon`. (`es9-daemon` change.)
- **Phase 3 — SuperDirt**. `gain` → `Dirt amp` when the OSC path is wired.

Files

- `purperl-tidal/src/Calypso/Prelude.purs` — `drum`, `DrumPart`, `On/DrumKit` instance (the surface change).
- `purperl-tidal/src/Tidal/Dispatcher.purs` — add `gain` (0..1)→velocity to the `MidiNote/MidiDrumKit` param lookup + `consumesVel`; later `GateDrumKit`.
- `purperl-tidal/src/Tidal/SessionWalker.purs` (+ `tidal_voice.erl`) — only if the compiled path must sample a `ValueMap` body (open question #2).
- `calypso/frontend/src/Calypso/Frontend/Tarot/Lower.purs` — emit `# gain` from `StepProfile` strength.
- `tarot-music` genres — unchanged; the strength is already there.

Risks / open questions

1. **Compiled-path params (#2 above)** — the one thing to settle before committing to Phase 1's shape. Verify how `play-piece` parts reach dispatch.
2. **drum type change** ripples through the `On` class + any `Erase/armPart` instance; needs a careful but mechanical pass + a full engine build.
3. **gain vs vel standardisation** — recommend `gain` neutral (0..1), `vel` absolute escape hatch. Confirm.
4. **Gate accent representation** (Phase 2) — accent gate vs velocity CV is an `es9-daemon/rig` decision.

Verification

Compile + play a swung, accented dembow / AfroCuban pattern; confirm velocity dynamics reach Ableton (drum-rack velocity layers respond) and that an unaccented genre is unchanged. Phase 0 gives an early read with zero code.